CANVIA

CAPACITACIÓN QA AUTOMATION · PARTE 1 DE 2

Pruebas de APIs con REST Assured y Cucumber

Qué es una API, cómo se prueba, cómo se escribe un escenario que negocio entiende y cómo se convierte en una prueba automática que corre en segundos.

Instructor **Giancarlo Palomino**

Java 17 · Maven · REST Assured 6 · Cucumber 7

Octubre 2026

LO QUE TE LLEVAS HOY

Cuatro bloques, de la idea al código

1

APIs sin misterio

Qué es una API, cómo es una petición HTTP, verbos y códigos de estado. Por qué probar aquí antes que en pantalla.

2

BDD y Gherkin

Escribir el comportamiento con ejemplos concretos, en un idioma que negocio, desarrollo y QA comparten.

3

REST Assured + Cucumber

Cómo una frase en español termina siendo una petición HTTP real con su verificación.

4

Ejercicios

Cinco retos progresivos sobre una API real. El último: cazar bugs que la API esconde a propósito.

REQUISITO

Tener el entorno del **Manual de instalación · Parte 1** en verde: `mvn test` → `BUILD SUCCESS`.

LA ANALOGÍA DEL RESTAURANTE

Una API es el mesero de un sistema

Tú no entras a la cocina. Le pides al mesero con un formato que él entiende (la carta), él lleva el pedido y vuelve con tu plato… o con un "no hay".

CLIENTE

App, web o prueba

"Quiero la reserva 42"

→

MESERO

API

recibe, valida y traduce el pedido

→

COCINA

Servidor y base de datos

busca la reserva y arma la respuesta

La carta = el contrato

Qué se puede pedir, con qué datos y qué se recibe. Suele estar en Swagger / OpenAPI.

El pedido = la petición

Viaja por HTTP con un método, una dirección, cabeceras y, a veces, un cuerpo JSON.

El plato = la respuesta

Un código de estado que resume el resultado y, normalmente, datos en JSON.

ANATOMÍA

Una petición y su respuesta, por dentro

Petición — lo que envías HTTP

```
POST /booking HTTP/1.1
Host: restful-booker.herokuapp.com
Content-Type: application/json
Accept: application/json

{
  "firstname": "Ana",
  "lastname": "Torres",
  "totalprice": 150,
  "depositpaid": true,
  "bookingdates": { "checkin": "2026-11-01", "checkout":
"2026-11-05" }
}
```

Respuesta — lo que recibes HTTP

```
HTTP/1.1 200 OK
Content-Type: application/json; charset=utf-8

{
  "bookingid": 4414,
  "booking": {
    "firstname": "Ana",
    "lastname": "Torres",
    "totalprice": 150,
    ...
  }
}
```

MÉTODO + RUTA

`POST /booking`: qué acción y sobre qué recurso.

CABECERAS

Metadatos: formato del cuerpo, formato aceptado, token.

CUERPO

Los datos, casi siempre en JSON.

CÓDIGO DE ESTADO

`200`: lo primero que verifica una prueba.

VERBOS HTTP

Cuatro verbos para el ciclo de vida de un dato

Casi toda API de negocio es un CRUD: crear, leer, actualizar y borrar. Cada acción tiene su verbo.

`POST`

Crear

`POST /booking`

Nueva reserva. Devuelve su id.

`GET`

Leer

`GET /booking/42`

Consulta sin modificar nada. Se puede repetir sin efectos.

`PUT · PATCH`

Actualizar

`PUT` reemplaza todo el recurso; `PATCH` cambia solo algunos campos.

`DELETE`

Borrar

`DELETE /booking/42`

En Restful Booker exige token.

PREGUNTA PARA LA SALA

Si haces `GET /booking/42` diez veces, ¿cuántas reservas cambian? ¿Y si haces `POST /booking` diez veces? Esa diferencia [idempotencia] decide qué pruebas pueden repetirse sin limpiar datos.

CÓDIGOS DE ESTADO

El primer dígito ya te cuenta la historia

2xx

Todo bien

`200` OK · `201` Created · `204` No Content

3xx

Ve a otro lado

`301` movido · `304` sin cambios. Poco común en APIs REST.

4xx

Error del cliente

`400` datos inválidos · `401` sin identificarse · `403` sin permiso · `404` no existe

5xx

Error del servidor

`500` algo se rompió adentro · `503` no disponible. **Siempre es un bug que reportar.**

HISTORIA REAL DE ESTE CURSO

Al preparar el material, una prueba esperaba `200` y recibió `418 I'm a teapot`. La causa: REST Assured enviaba una cabecera `Accept` con cuatro formatos y la API solo acepta uno. **Un código raro casi nunca es un misterio: es la API diciendo exactamente qué no le gustó.**

REGLA DE QA

Un `4xx` puede ser correcto (pediste algo prohibido). Un `5xx` nunca lo es: la API no supo manejar tu pedido.

ESTRATEGIA

¿Por qué probar la API antes que la pantalla?



Velocidad

Una prueba de API tarda milisegundos; la misma por pantalla, segundos. En este curso: 4 escenarios en menos de 5 s.

Estabilidad

No depende de botones que se mueven, animaciones ni navegadores.

Temprano

La API suele estar lista antes que la pantalla: encuentras bugs cuando son baratos de arreglar.

02

BDD y Gherkin

Antes de automatizar, hay que ponerse de acuerdo en qué significa “funciona”.

BEHAVIOUR-DRIVEN DEVELOPMENT

BDD es una conversación antes que una herramienta

Tres miradas revisan una historia y la aterrizan en ejemplos concretos. Esos ejemplos se escriben en Gherkin y se convierten en pruebas.

Negocio

"Un huésped puede reservar si indica nombre y fechas."

Aporta: el porqué y las reglas.

Desarrollo

"¿Y si la salida es anterior a la llegada? ¿Precio negativo?"

Aporta: límites técnicos.

QA

"Entonces: fechas invertidas → 400 con mensaje claro."

Aporta: casos borde y cómo verificarlos.

1
Historia
lo que se quiere

→

2
Ejemplos
casos concretos con datos

→

3
Escenarios Gherkin
documentación viva

→

4
Prueba automática
verifica cada día

GHERKIN

Un archivo .feature, línea por línea

reservas.feature — GHERKIN

```gherkin
# language: es
Característica: Gestión de reservas
  Como cliente de la API de reservas
  Quiero crear y consultar reservas
  Para gestionar las estadías de los huéspedes

  Antecedentes:
    Dado que la API de reservas está disponible

  @smoke
  Escenario: Consultar una reserva recién creada
    Dado que existe una reserva a nombre de "Rosa"
    Cuando consulto esa reserva
    Entonces el código de respuesta es 200
    Y la respuesta contiene el campo "firstname"
```

LANGUAGE: ES
Palabras clave en español.

CARACTERÍSTICA + HISTORIA
El tema del archivo: quién, qué y para qué.

ANTECEDENTES
Se ejecuta antes de cada escenario del archivo.

@SMOKE
Etiqueta para filtrar qué ejecutar.

ESCENARIO
Un ejemplo concreto. Debe poder leerse solo, sin depender de otro escenario.

LA ESTRUCTURA DE TODA PRUEBA

Dado · Cuando · Entonces

Es el mismo patrón de cualquier buena prueba (preparar, actuar, verificar), escrito para que una persona de negocio lo lea en voz alta.

D

Dado · el contexto

El estado del mundo antes de actuar.

Dado que existe una reserva a nombre de "Rosa"

Técnicamente: crear los datos que la prueba necesita.

C

Cuando · la acción

Una sola acción del usuario o del sistema.

Cuando consulto esa reserva

Técnicamente: la petición HTTP que se prueba.

E

Entonces · el resultado

Lo observable que debe ocurrir.

Entonces el código de respuesta es 200

Técnicamente: las aserciones.

Y · PERO

Encadenan pasos del mismo tipo: *Entonces el código es 200* ***Y*** *la respuesta contiene el campo "firstname"*. Más de un **Cuando** en un escenario suele indicar que hay dos escenarios mezclados.

EL ERROR MÁS COMÚN

Escribe el qué, no el cómo

IMPERATIVO · EVITAR

Atado a la técnica ✗

```
Escenario: Crear reserva
  Dado que la URL base es "https://restful-booker..."
  Y que agrego la cabecera "Content-Type" "application/json"
  Y que el body tiene "firstname" igual a "Ana"
  Cuando hago POST a "/booking"
  Entonces el status es 200
  Y el JSONPath "booking.firstname" es "Ana"
```

Negocio no lo entiende y cualquier cambio técnico rompe el texto.

DECLARATIVO · PREFERIR

Habla de negocio ✓

```
Escenario: Un huésped reserva su estadía
  Cuando creo una reserva para "Ana" "Torres" por 150 soles
  Entonces el código de respuesta es 200
  Y el nombre de la reserva creada es "Ana"
```

URL, cabeceras y JSONPath viven en el código de los steps, donde pueden cambiar sin tocar el escenario.

03

REST Assured + Cucumber

De una frase en español a una petición HTTP real con su verificación.

EL VIAJE DE UNA FRASE

Cómo se conecta todo

1 · FEATURE
Gherkin
`Cuando consulto esa reserva`

→

2 · CUCUMBER
Busca el step
compara la frase con las anotaciones

→

3 · STEP DEFINITION
Método Java
`@Cuando("consulto esa reserva")`

→

4 · REST ASSURED
HTTP real
GET /booking/4414

→

5 · RESULTADO
Verde o rojo
aserción + reporte HTML

Runner

`RunCucumberTest` : le dice a JUnit dónde están los features y el código (*glue*).

Hooks

`@Before` prepara cada escenario: URL base, registro de peticiones si falla.

Contexto

La mochila del escenario: lo que un paso guarda (la respuesta, el id) otro lo lee.

LO QUE CORRE AL ESCRIBIR MVN TEST

Maven → Surefire → JUnit Platform → motor de Cucumber → tus steps → REST Assured → la API. Si algo falla, el mensaje te dice en cuál de estos eslabones fue.

LA SINTAXIS DE REST ASSURED

given · when · then, también en el código

ReservaSteps.java — JAVA

```java
contexto.respuesta = given()                    // preparar
        .contentType(ContentType.JSON)
        .accept("application/json")
        .body(cuerpo)                           // Map → JSON (Jackson)
    .when()                                     // actuar
        .post("/booking");

contexto.respuesta.then()                       // verificar
        .statusCode(200)
        .body("bookingid", notNullValue())
        .body("booking.firstname", equalTo("Ana"))
        .time(lessThan(2000L));                 // y que sea rápida
```

GIVEN()

Todo lo que acompaña a la petición: cabeceras, parámetros, cuerpo, autenticación.

WHEN()

El verbo y la ruta: `get`, `post`, `put`, `patch`, `delete`.

THEN()

Las verificaciones. Si una falla, la prueba se pone en rojo con un mensaje claro.

POR QUÉ GUARDAMOS LA RESPUESTA

El paso *Cuando* hace la petición y el paso *Entonces* la verifica: son métodos distintos, por eso la respuesta viaja en el `Contexto`.

LEER EL JSON

Rutas para llegar a cualquier dato

Respuesta de POST /booking — JSON

```
{
  "bookingid": 4414,
  "booking": {
    "firstname": "Ana",
    "lastname": "Torres",
    "totalprice": 150,
    "depositpaid": true,
    "bookingdates": {
      "checkin": "2026-11-01",
      "checkout": "2026-11-05"
    }
  }
}
```

RUTA	VALOR
`bookingid`	`4414`
`booking.firstname`	`"Ana"`
`booking.bookingdates.checkin`	`"2026-11-01"`
`booking.totalprice`	`150`

MATCHERS DE HAMCREST MÁS USADOS

`equalTo(x)` · `notNullValue()` · `greaterThan(0)` · `hasSize(3)` · `containsString("An")` · `hasKey("token")`

EXTRAER PARA USAR DESPUÉS

`int id = respuesta.then().extract().path("bookingid");` — así un paso obtiene el id que otro necesita.

COMPARTIR DATOS ENTRE PASOS

La mochila del escenario

Cada escenario recibe una mochila nueva y vacía. Los pasos guardan y sacan cosas de ella. Al terminar el escenario, se descarta: el siguiente empieza limpio.

Contexto.java + constructor — JAVA

```java
public class Contexto {
    public Response respuesta;
    public Integer idReserva;
}

public class ReservaSteps {
    private final Contexto contexto;

    // PicoContainer crea el Contexto y lo inyecta aquí
    public ReservaSteps(Contexto contexto) {
        this.contexto = contexto;
    }
}
```

DADO
guarda
idReserva = 4414

→

CUANDO
usa y guarda
GET /booking/4414 → respuesta

→

ENTONCES
verifica
respuesta.statusCode(200)

TENTACIÓN A EVITAR

Variables `static` para compartir datos. Funcionan… hasta que dos escenarios corren en paralelo y se pisan los datos.

POR QUÉ NO CONSULTAMOS LA RESERVA 1

La API es pública: otro alumno puede borrarla. **Cada escenario crea los datos que necesita.** Prueba independiente = prueba confiable.

PRUEBAS GUIADAS POR DATOS

Un escenario, muchas pruebas

Esquema del escenario — GHERKIN

```
@regresion
Esquema del escenario: Crear reservas con distintos huéspedes
  Cuando creo una reserva para "<nombre>" "<apellido>" por <precio>
soles
  Entonces el código de respuesta es 200
  Y el nombre de la reserva creada es "<nombre>"

  Ejemplos:
    | nombre | apellido | precio |
    | Ana    | Torres   | 150    |
    | Luis   | Quispe   | 320    |
    | María  | Huamán   | 0      |
```

CÓMO FUNCIONA

Cucumber reemplaza cada `<columna>` con el valor de la fila. Tres filas = tres pruebas independientes en el reporte.

ELIGE FILAS QUE ENSEÑEN ALGO

No sirve repetir el mismo caso con otros nombres. Cada fila debe probar algo distinto: un límite (precio 0), un carácter especial (tildes, ñ), un valor máximo.

TABLA DE DATOS (DATA TABLE)

Para enviar muchos campos en un solo paso, usa una tabla de dos columnas bajo el paso; en Java llega como `Map<String, String>`. Lo practicarás en el ejercicio 2.

TAGS Y COMANDOS

Ejecutar lo que necesitas, cuando lo necesitas

COMANDO (WINDOWS, MACOS Y LINUX)	QUÉ CORRE
`mvn test`	Todo
`mvn test "-Dcucumber.filter.tags=@smoke"`	Lo crítico, en segundos
`mvn test "-Dcucumber.filter.tags=@regresion and not @lento"`	Combinaciones
`mvn test "-DbaseUrl=https://qa.miapi.com"`	Otro ambiente

UNA ESTRATEGIA DE TAGS SENCILLA

`@smoke`: lo mínimo que debe funcionar tras cada despliegue. `@regresion`: todo. `@wip`: lo que estás escribiendo ahora.

LAS COMILLAS NO SON DECORACIÓN

En PowerShell, sin las comillas que envuelven `-D…`, Maven responde *Unknown lifecycle phase*. Con ellas, el comando es el mismo en los tres sistemas.

LA EVIDENCIA

`target/cucumber-report.html`: el reporte que se adjunta a la historia o al pipeline.

LEER EL ROJO

Cuatro errores que verás esta semana

`step undefined`

La frase no tiene código

Una tilde o palabra distinta entre el feature y la anotación. Cucumber te propone el método a crear: cópialo.

`Expected <200> but was <404>`

La aserción falló

El código se ejecutó bien, pero la API respondió otra cosa. Mira el log de petición y respuesta que REST Assured imprime.

`no JSON serializer found`

Falta una librería

Sin Jackson, REST Assured no sabe convertir un `Map` a JSON. Se arregla en el `pom.xml`.

`418 I'm a teapot`

La API rechaza tu formato

Compara tus cabeceras con las de `curl` o Postman. La diferencia suele estar en `Accept` o `Content-Type`.

MÉTODO DE 3 PASOS

1. ¿En qué paso falló? **2.** ¿Es aserción (la API respondió distinto) o excepción (el código no pudo ejecutarse)? **3.** Reproduce la petición a mano con curl o Postman antes de tocar el código.

04

Ejercicios

Cinco retos sobre `restful-booker.herokuapp.com`. Cada uno parte del proyecto base y suma una habilidad. El quinto no tiene respuesta única: hay que investigar.

01 · 02
Consultar, crear y verificar todos los datos.

03
Autenticación con token.

04
Ciclo de vida completo de una reserva.

05
Caza de bugs, contrato y rendimiento.

1
NIVEL

CALENTAMIENTO

Buscar reservas y pedir lo que no existe

25 min

Dificultad

OBJETIVO

Escribir dos escenarios nuevos reutilizando todo lo que puedas de los steps existentes.

CRITERIOS DE ACEPTACIÓN

1. Crear una reserva a nombre de un huésped con un apellido único y encontrarla con `GET /booking?lastname=…` : la lista no está vacía y contiene su id.
2. Consultar `GET /booking/999999999` responde **404**.
3. Ambos escenarios llevan la etiqueta `@ejercicio1` y corren con el filtro de tags.
4. Ningún escenario depende de datos que no haya creado él mismo.

Punto de partida — GHERKIN

```gherkin
@ejercicio1
Escenario: Encontrar una reserva por apellido
  Dado que existe una reserva a nombre de "Rosa" "Q-1739"
  Cuando busco reservas con el apellido "Q-1739"
  Entonces el código de respuesta es 200
  Y la lista contiene la reserva creada

@ejercicio1
Escenario: Consultar una reserva inexistente
  Cuando consulto la reserva con id 999999999
  Entonces el código de respuesta es 404
```

PISTAS

`given().queryParam("lastname", apellido)` · la respuesta es una lista: `body("bookingid", hasItem(contexto.idReserva))` · genera un apellido único con `System.currentTimeMillis()` si lo prefieres.

Aprendes: query params (`queryParam`), validar listas con `hasItem`, escenarios negativos.

2
NIVEL

DATOS COMPLETOS

Crear con una tabla y verificar cada campo

35 min
Dificultad

OBJETIVO

Enviar todos los datos de la reserva desde una **Data Table** y comprobar que la API guardó exactamente lo que se envió, consultándola después.

CRITERIOS DE ACEPTACIÓN

1. El paso recibe la tabla como `Map<String, String>` y arma el cuerpo con fechas anidadas.
2. Tras crearla, un `GET` devuelve los mismos 6 valores (nombre, apellido, precio, depósito, llegada, salida).
3. Si un valor difiere, el mensaje de error dice qué campo y qué se esperaba.
4. Un *Esquema* repite la prueba con un nombre con tilde y uno con ñ.

Aprendes: Data Tables, construir JSON anidado, conversión de tipos (`"150"` → `150`), verificar persistencia y no solo la respuesta.

Punto de partida — GHERKIN + JAVA

```
@ejercicio2
Escenario: La API guarda exactamente lo enviado
  Cuando creo una reserva con los datos:
    | firstname   | Ñusta      |
    | lastname    | Mamani     |
    | totalprice  | 480        |
    | depositpaid | true       |
    | checkin     | 2026-12-20 |
    | checkout    | 2026-12-27 |
  Y consulto esa reserva
  Entonces la reserva guardada coincide con los datos enviados

@Cuando("creo una reserva con los datos:")
public void creoConDatos(Map<String, String> datos) { ... }
```

PISTA

Guarda el `Map` en el `Contexto` para compararlo en el *Entonces*. Para precio y depósito usa `Integer.parseInt` y `Boolean.parseBoolean`.

3
NIVEL

SEGURIDAD

Autenticarse y actualizar con token

40 min
Dificultad

OBJETIVO

Obtener un token con `POST /auth` (usuario `admin`, clave `password123`) y usarlo para modificar una reserva con `PUT`.

CRITERIOS DE ACEPTACIÓN

1. Un paso *Dado que estoy autenticado como administrador* guarda el token en el Contexto.
2. Con token, `PUT /booking/{id}` cambia el precio y responde **200** con el valor nuevo.
3. Sin token, la misma petición responde **403**. Con un token inventado, también.
4. Usuario y clave no están escritos en el código: se leen de `System.getProperty` o variables de entorno.

Punto de partida — GHERKIN

```gherkin
@ejercicio3 @seguridad
Escenario: El administrador actualiza el precio
  Dado que estoy autenticado como administrador
  Y que existe una reserva a nombre de "Carlos"
  Cuando actualizo el precio de esa reserva a 999
  Entonces el código de respuesta es 200
  Y el precio de la reserva es 999

@ejercicio3 @seguridad
Esquema del escenario: Nadie más puede actualizar
  Dado que existe una reserva a nombre de "Carlos"
  Cuando actualizo el precio con el token "<token>"
  Entonces el código de respuesta es 403
  Ejemplos:
    | token          |
    |                |
    | token-inventado |
```

PARA PENSAR

Prueba también `POST /auth` con una clave incorrecta. ¿Qué código de estado devuelve? ¿Es el que esperarías? Anótalo: lo usarás en el ejercicio 5.

Aprendes: autenticación por token, cookies (`.cookie("token", token)`), PUT vs PATCH, pruebas negativas de seguridad, no versionar credenciales.

4
NIVEL

FLUJO COMPLETO

El ciclo de vida de una reserva

45 min
Dificultad ●●●●○

OBJETIVO

Automatizar un flujo de punta a punta: crear → modificar parcialmente → borrar → confirmar que ya no existe. Y además, **limpiar** lo que creas.

CRITERIOS DE ACEPTACIÓN

1. `PATCH` cambia solo `additionalneeds`; el resto de campos queda igual (verifícalo).
2. Tras `DELETE`, un `GET` a ese id responde **404**.
3. Un hook `@After("@limpieza")` borra las reservas que el escenario creó, aunque el escenario falle.
4. Separa los steps en dos clases (`ReservaSteps`, `AutenticacionSteps`) que comparten el mismo `Contexto`.

Aprendes: PATCH, hooks condicionados por tag, limpieza de datos de prueba, organizar steps en varias clases.

Punto de partida — GHERKIN

```
@ejercicio4 @limpieza
Escenario: Una reserva nace, cambia y desaparece
  Dado que estoy autenticado como administrador
  Y que existe una reserva a nombre de "Elena"
  Cuando cambio sus necesidades adicionales a "Cuna"
  Entonces la reserva conserva sus demás datos
  Cuando elimino esa reserva
  Entonces el código de respuesta es 201
  Y la reserva ya no existe
```

DISCUSIÓN OBLIGATORIA

Restful Booker responde **201 Created** al borrar. Tu prueba pasa… ¿pero es correcto? ¿Qué código esperarías tú? ¿Lo reportarías como bug o lo aceptarías como contrato? Escribe tu respuesta en el PR.

NOTA

Este escenario tiene dos *Cuando*: es un flujo E2E a propósito. En pruebas de regla de negocio, preferimos uno.

5
RETO

PENSAR COMO QA

Caza de bugs: la API esconde al menos cinco

60 min
Dificultad ●●●●●

OBJETIVO

Restful Booker tiene comportamientos que ninguna API de producción debería tener. Encuéntralos, escribe un escenario que **describa lo correcto** (y que hoy falle) y redacta el reporte del bug.

DÓNDE BUSCAR

1. Reglas de negocio: ¿acepta un precio negativo? ¿Una salida anterior a la llegada?
2. Validación: ¿qué pasa si falta un campo obligatorio en el cuerpo?
3. Seguridad: ¿qué código devuelve `/auth` con credenciales malas?
4. Semántica HTTP: ¿qué código devuelve un borrado exitoso?

Aprendes: diseño de pruebas negativas, valores límite, redactar bugs, etiquetar con `@bug` para no bloquear el pipeline.

Y PARA SUBIR DE NIVEL

Contrato con JSON Schema

Agrega `io.rest-assured:json-schema-validator` y valida la respuesta con `matchesJsonSchemaInClasspath("schemas/reserva.json")`: tipos, campos obligatorios y formato de fecha.

Especificación reutilizable

Crea un `RequestSpecification` con URL, cabeceras y log; que ningún step repita `contentType` ni `accept`.

Rendimiento básico

Un paso *Y la respuesta llega en menos de 2 segundos* con `time(lessThan(2000L))`.

ENTREGABLE

Escenarios con `@bug` + un archivo `BUGS.md`: título, pasos, resultado esperado, resultado obtenido, severidad.

CÓMO SE REVISA

Entrega y rúbrica

CRITERIO	PESO	QUÉ SE MIRA
Funciona	40 %	`mvn test` en verde (salvo los `@bug` documentados) en una máquina limpia.
Escenarios	25 %	Declarativos, independientes, un solo propósito, datos propios.
Código	20 %	Sin duplicar steps, sin `static` para estado, sin credenciales en el código.
Criterio QA	15 %	Casos negativos y límite; calidad del `BUGS.md`.

CÓMO ENTREGAR

Una rama por ejercicio (`feature/ejercicio-3`), un PR con el reporte HTML adjunto y commits en español: `feat: agrega escenario de autenticación` .

LO QUE MÁS RESTA

Escenarios que dependen del orden de ejecución o de datos que no crearon. Si tu prueba pasa sola y falla con las demás, no está terminada.

CANVIA

PARA LLEVARTE

Tres ideas que valen más que cualquier librería

1 · Acordar antes de automatizar
Un escenario es un acuerdo escrito con ejemplos. Si negocio no lo entiende, no es BDD.

2 · Cada prueba, sus datos
Independiente, repetible y limpia. Así puede correr mil veces sin mentir.

3 · El rojo informa
Lee el mensaje completo: la API casi siempre dice qué pasó.

Siguiente sesión: **Parte 2 · Playwright + TypeScript + Cucumber**

Documentación: **rest-assured.io** · **cucumber.io/docs**